

6 Unit Testing

Unit testing-ul este indispensabil in dezvoltarea proiectelor moderne. Au fost dezvoltate utilitare gratuite de testare a claselor, care au contribuit la cresterea vitezei de programare si la micsorarea semnificativa a numarului de bug-uri.

6.1 JUnit

JUnit este un framwork (bibliotecă) popular in realizarea testelor unitare (unit tests). Alte proiecte (biblioteci) similare sunt Mockito, PowerMock, TestNG (derivat din JUnit).

Printre avantajele folosirii JUnit sunt:

- clasele de test sunt usor de scris si modificat pe masura ce codul sursa se maresc, putand fi compilate impreuna cu codul sursa al proiectului
- clasele de test JUnit pot fi rulate automat (in suita), rezultatele fiind vizibile imediat
- clasele de test maresc increderea programatorului in codul sursa scris si ii permit sa urmareasca mai usor cerintele de implementare ale proiectului
- testele pot fi scrise inaintea implementarii, fapt ce garanteaza intelegerea functionalitatii de catre dezvoltator

6.2 Dependinte

Pentru a rula testele unitare trebuie adaugate bibliotecile aferente. Dependintele necesare pentru a rula teste unitare cu JUnit 5 sunt:

pom.xml (pentru proiect tip maven):

```
<dependencies>
  <dependency>
    <!-- JUnit Jupiter is the API for writing tests using JUnit 5. -->
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <!-- This Bill of Materials POM can be used to ease dependency management when
    referencing multiple JUnit artifacts using Gradle or Maven. -->
    <groupId>org.junit</groupId>
    <artifactId>junit-bom</artifactId>
    <version>5.10.0</version>
    <type>pom</type>
    <scope>test</scope>
  </dependency>
</dependencies>
```

build.gradle (pentru proiect gradle format groovy)

```
dependencies {
    testImplementation platform('org.junit:junit-bom:5.10.0')
    testImplementation 'org.junit.jupiter:junit-jupiter'
}
```

Laborator 6

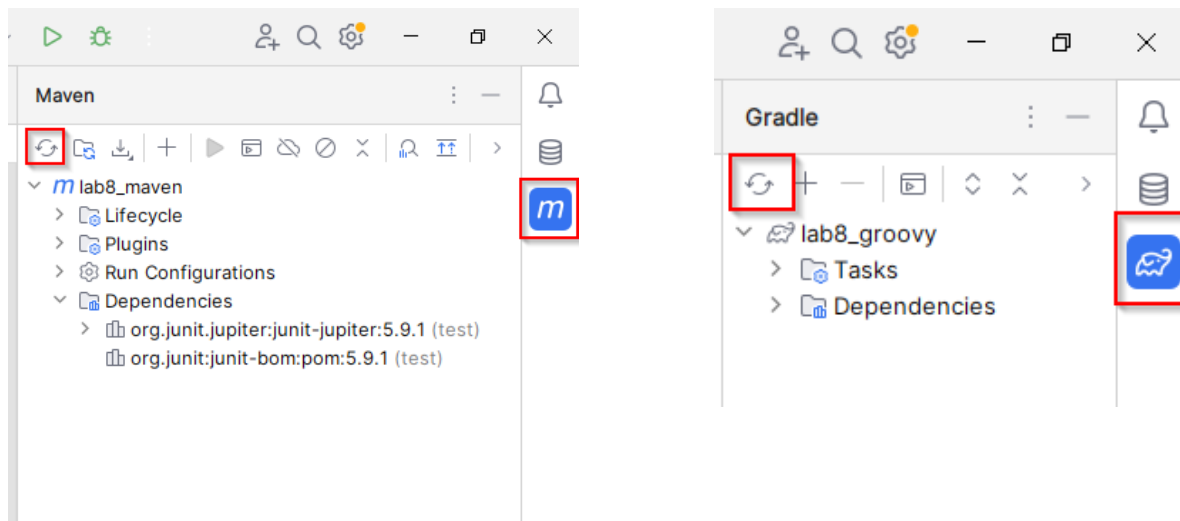
build.gradle.kts (pentru proiect gradle format kotlin)

```
dependencies {
    testImplementation(platform("org.junit:junit-bom:5.10.0"))
    testImplementation("org.junit.jupiter:junit-jupiter")
}
```

Pentru situatia in care nu aveti adaugate deja aceste dependinte, adaugarea lor se poate face manual adaugand (copy&paste) detaliile de mai sus sau folosindu-ne de facilitatile IntelliJ IDEA.

Pentru adaugare, editati fisierul pom.xml / build.gradle – click dreapta: Generate > Add Maven artifact dependency > org.junit.jupiter, si apoi org.junit:junit-bom.

Dupa aceea reload maven project/gradle project:



Dupa acest moment fisierele *.jar necesare au fost downloadate si vor putea fi folosite la compilare si rulare teste unitare.

6.3 Utilizarea framework-ului: anotari

Cel mai mare avantaj al JUnit este viteza rapida de scriere a testelor, iar lucrul acesta este posibil datorita adnotarilor. Anotarile sunt termeni standardizati, prefatati de semnul "@", plasati fix inaintea semnaturii unei functii. Scopul acestora este ca, in momentul compilarii, compilatorul sa stie sa adauge functionalitate in plus metodei.

Printre anotarile de baza din JUnit5 se numara:

- **@Test** - metoda va functiona ca test, aceasta va trebui sa intoarca o valoare de true daca functionalitatea testata functioneaza in modul dorit, False in mod contrar (lucru posibil prin assert-uri, despre care vom vorbi mai jos)
- **@BeforeEach** - metoda va fi executata inaintea fiecarui test
- **@AfterEach** - metoda va fi executata dupa fiecare test
- **@DisplayName("Some_String")** - se foloseste impreuna cu @Test; cand testul se ruleaza, la consola va aparea la output cu numele "Some_String"

6.4 Utilizarea framework-ului: assert

Assert se refera la o un set de metode statice, care sunt gasite in clasa org.junit.jupiter.api.Assertions , afirma valoarea de adevar a diferite expresii. In continuare, va vom prezenta cateva exemple de assert-uri des folosite:

Laborator 6

- **assertEquals(value_1, value_2)** - verifica daca cele doua valori sunt egale
- **assertTrue(boolean value)**, respectiv **assertFalse(boolean value)** - verifica daca value este True, respectiv False
- **assertNull(obj)**, respectiv **assertNotNull(obj)** - verifica daca obj este null, respectiv daca nu este null

6.5 Scrierea testelor unitare

Pentru a scrie teste corect si cu buna vizibilitate, aveti in minte urmatoarele lucruri:

- puneti nume sugestive pentru fiecare metoda de test pe care o implementati
- scrieti metode de test care sa testeze doar o anumita functionalitate, nu mai multe deodata
- tineti minte ca testele sunt facute sa verifice ca implementarile facute de voi functioneaza in modul dorit de voi
- principul AAA (Arrange-Act-Assert)

6.6 Arrange – Act – Assert

Anatomia metodei care constituie un test unitar se bazeaza pe idei principale care trebuie acoperite de liniile de cod: Arrange – act – assert.

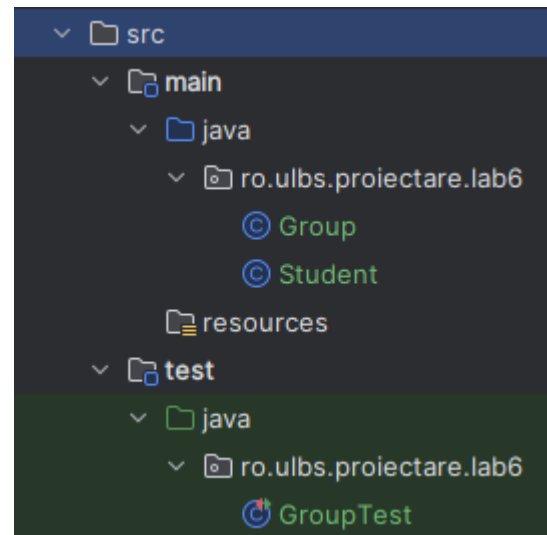
Liniile de cod care constituie arrange se refera la faptul ca acestea vor instantia si vor seta acele proprietati necesare pentru rulara testului curent.

Liniile de cod care se refera la 'act' vor apela functia pe care o testam. De obicei aceasta este constituita dintr-o singura linie de cod.

Liniile de cod care constituie zona 'assert' sunt cele care verifica una sau mai multe proprietati afectate de executia liniei 'act'.

8.7 Exemplu

Vom folosi in exemplul de mai jos, clasele Student si Group (in pachetul ro.ulbs.proiectare.lab6) si clasa GroupTest (in acelasi pachet) in ierarhia de teste. Structura este urmatoarea:



Laborator 6

Sursele:

```
package ro.ulbs.proiectare.lab6;
public class Student {
    private String name;
    private int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}
```

```
package ro.ulbs.proiectare.lab6;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
public class Group {
    private List<Student> students;

    public Group () {
        students = new ArrayList<Student>();
    }

    public List<Student> getStudents() {
        return students;
    }

    public void setStudents(List<Student> students) {
        this.students = students;
    }

    public void addStudent(Student student) {
        students.add(student);
    }

    public boolean removeStudent(Student student) {
        Iterator<Student> it = students.iterator();
```

Laborator 6

```
        boolean removedAll = false;
        while (it.hasNext()) {
            Student st = it.next();
            if (null != st.getName() && st.getName().equals(student.getName())) {
                it.remove();
                removedAll = true;
            }
        }
        return removedAll;
    }

    public Student getStudent(String name) {
        for (Student st : students) {
            if (null != st.getName() && st.getName().equals(name)) {
                return st;
            }
        }
        return null;
    }

    public boolean areStudentsInGroup() {
        if (0 == students.size()) {
            return false;
        }
        return true;
    }
}
```

Laborator 6

```

package ro.ulbs.proiectare.lab6;
import org.junit.jupiter.api.*;
public class GroupTest {
    private Group group;

    @BeforeEach // aceasta adnotare determina ca metoda de mai jos sa se execute inaintea
    fiecarui @Test
    public void setup() {
        group = new Group();
    }

    @Test
    public void testNoStudentInGroup() {
        Assertions.assertEquals(false, group.areStudentsInGroup());
    }

    @Test
    public void testAddStudent() {
        //arrange
        Student st = new Student("Elena", 19);

        //act
        group.addStudent(st);

        //assert
        Assertions.assertTrue(group.getStudent("Elena").equals(st));
    }

    @Test
    public void testRemoveStudentExisting() {
        //arrange
        group.addStudent(new Student("Ioan", 21));
        group.addStudent(new Student("Elena", 19));
        group.addStudent(new Student("Mihaela", 22));

        Student stud4 = new Student("Elena", 19);

        //act
        boolean result = group.removeStudent(stud4);

        //assert
        Assertions.assertTrue(result);
        Assertions.assertEquals(group.getStudents().size(), 2);
    }

    @Test
    public void testRemoveStudentNotExisting() {
        //arrange
        group.addStudent(new Student("Ioan", 21));
        group.addStudent(new Student("Elena", 19));
        group.addStudent(new Student("Mihaela", 22));

        Student stud4 = new Student("Radu", 20);

        //act
        boolean result = group.removeStudent(stud4);
    }
}

```

Laborator 6

```

        //assert
        Assertions.assertFalse(result);
        Assertions.assertEquals(group.getStudents().size(), 3);
    }

    @Test
    public void testNullList() {
        //arrange
        group.addStudent(new Student("Ioan", 21));
        group.setStudents(null);

        Exception exception = Assertions.assertThrows(NullPointerException.class, () -> {
            //act
            group.addStudent(new Student("Elena", 19));
        });

        //assert
        Assertions.assertNotNull(exception);
    }

    @AfterEach //metoda este executata la sfarsitul fiecarui test
    public void tearDown() {
        group = null;
    }
}

```

- fiecare metoda de test este marcata de adnotarea: `@Test`

- metodele de `setUp` (avand adnotarea: `@BeforeEach`) si `tearDown` (avand adnotarea: `@AfterEach`) se apeleaza inainte, respectiv dupa fiecare test. Se intrebuinteaza pentru initializarea/eliberarea resurselor ce constituie mediul de testare, evitandu-se totodata duplicarea codului si respectandu-se principiul de independenta a testelor. Pentru exemplul dat ordinea este:

```

@BeforeEach setUp
@Test testNoStudentInGroup
@AfterEach tearDown
@BeforeEach setUp
@Test testAddStudent
@AfterEach tearDown

```

- in contextul mostenirii daca `ChildTest` extends `ParentTest` ordinea de executie este urmatoarea:

```

ParentTest @BeforeEach setUp
ChildTest @BeforeEach setUpSub
ChildTest @Test testChild1
ChildTest @AfterEach tearDownSub
ParentTest @AfterEach tearDown

```

- pentru compararea rezultatului asteptat cu rezultatul curent se folosesc apeluri `assert`:

```

assertTrue
assertFalse
assertEquals

```

Laborator 6

assertNull / assertNotNull

Rulare teste:

- selectati sagetutele verzi din dreptul clasei – se vor executa toate metodele test. Selectand sagetuta din dreptul unei metode de test va rula doar acea metoda
- Click dreapta clasa test → Run 'YourClassName'

Daca toate testele unitare se executa cu success rezultatul este:

The screenshot shows the IntelliJ IDEA Run window for a Gradle project. The 'Test Results' tab is active, showing a green checkmark and the text '5 tests passed 5 tests total, 33 ms'. Below this, a list of tasks is displayed: > Task :compileJava, > Task :processResources NO-SOURCE, > Task :classes, > Task :compileTestJava, > Task :processTestResources NO-SOURCE, > Task :testClasses, > Task :test. The build is successful, with the message 'BUILD SUCCESSFUL in 1s' and '3 actionable tasks: 3 executed'. A link to the Gradle documentation is provided at the bottom: 'Consider enabling configuration cache to speed up this build: <https://docs.gradle.org/9.2>'.

Daca vreun test este fara success atunci:

The screenshot shows the IntelliJ IDEA Run window for a Gradle project. The 'Test Results' tab is active, showing a red X icon and the text '1 test failed, 4 passed 5 tests total, 45 ms'. Below this, a list of tasks is displayed: > Task :compileJava UP-TO-DATE, > Task :processResources NO-SOURCE, > Task :classes UP-TO-DATE, > Task :compileTestJava, > Task :processTestResources NO-SOURCE, > Task :testClasses. The build failed with the message 'Cannot invoke "Object.equals(Object)" because the return value of "ro.ulbs.proiectare.lab6.Group.getStudent(String)" is null'. The exception is a 'java.lang.NullPointerException'. A link to the Gradle documentation is provided at the bottom: 'Consider enabling configuration cache to speed up this build: <https://docs.gradle.org/9.2>'.

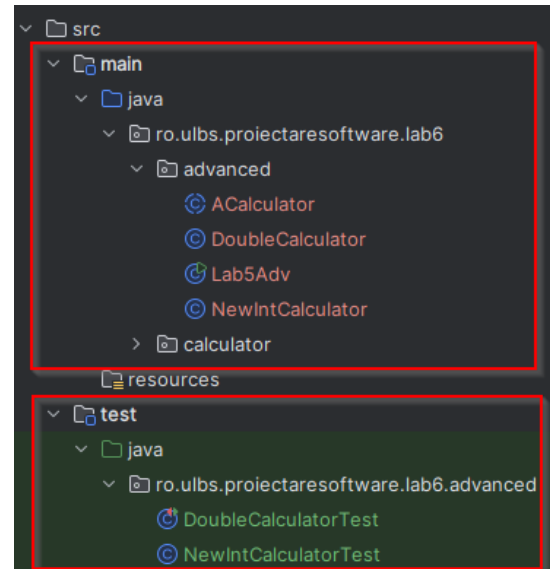
6.8 Probleme

6.8.1 Deschideti proiectul ProiectareSoftware si adaugati sursele din arhiva **laborator6_starter_src.zip**

Laborator 6

Pentru clasele NewIntCalculator si DoubleCalculator creati clasele de test NewIntCalculatorTest si DoubleCalculatorTest:

Realizati testele unitare pentru fiecare metoda existenta in aceste clase, realizand teste unitare cu varii parametrii de intrare. Implementati conceptul arrange – act – assert.



6.8.2 Pentru a testa robustetea codului in conditii neplanificate cum ar fi inmultirea si impartirea cu 0 realizati teste unitare care sa verifice comportamentul in conditii exceptionale (negative path). Astfel determinati ce exceptii sunt aruncate de codul de mai sus, si apoi cum ar trebui sa se comporte codul.

6.8.3 Deschideti proiectul Students, si

- creati o noua clasa **AplicatieCuBursa** in care vom defini un nou main:

```
public class AplicatieCuBursa {

    static void main(String[] args) {
        AplicatieCuBursa instanta = new AplicatieCuBursa();
        List<StudentBursier> lista = instanta.genereaza();
        for (StudentBursier student : lista) {
            System.out.println(student);
        }
        System.out.println("-----");
        List<StudentBursier> sortata = instanta.sorteaza(lista);
        for (StudentBursier student : sortata) {
            System.out.println(student);
        }
    }

    public List<StudentBursier> genereaza() {
        List<StudentBursier> lista = new ArrayList<>();
        lista.add( new StudentBursier(1025,"Andrei","Popa","ISM141/2", 8.70, 725.50));
        lista.add( new StudentBursier(1024,"Ioan","Mihalcea","ISM141/1", 9.80, 801.10));
        lista.add( new StudentBursier(1029,"Bianca","Popescu","TI131/1", 9.10, 780.80));
        lista.add( new StudentBursier(1026,"Anamaria","Prodan","TI131/1", 8.90, 745.50));
        lista.add( new StudentBursier(1029,"Bianca","Popescu","TI131/1", 9.10, 100.00));
        return lista;
    }
}
```

Laborator 6

```

    public List<StudentBursier> sorteaza(List<StudentBursier> lst) {
        // aici implementati logica pentru sortare:
        // Comparati formatia de studiu, apoi numele, apoi prenumele, apoi nota, apoi
        cuantumul bursei
        // apoi returnati lista.
    }
}

```

Asigurati-va ca aveti getBursa in clasa StudentBursier:

```

public class StudentBursier extends Student {
    double cuantumBursa;

    public StudentBursier(int numarMatricol, String prenume, String nume, String
    formatieDeStudiu, double nota, double bursa) {
        super(numarMatricol, prenume, nume, formatieDeStudiu);
        super.nota = nota;
        this.cuantumBursa = bursa;
    }

    public double getCuantumBursa() {
        return cuantumBursa;
    }

    @Override
    public String toString() {
        String s = super.toString();
        s += String.format("    [ %6.2f ]", cuantumBursa);
        return s;
    }

    @Override
    public boolean equals(Object o) {
        if (o == null || getClass() != o.getClass()) return false;
        if (!super.equals(o)) return false;
        StudentBursier that = (StudentBursier) o;
        return Double.compare(cuantumBursa, that.cuantumBursa) == 0;
    }

    @Override
    public int hashCode() {
        return Objects.hash(super.hashCode(), cuantumBursa);
    }
}

```

Creati clasa de test unitar pentru AplicatieCuBursa, in care sa realizati test pentru metoda sorteaza()

6.9 Referinte

IntelliJ IDEA - adaugare dependinte in build.gradle (Proiect Gradle, folosind forma Groovy)

https://www.jetbrains.com/help/idea/work-with-gradle-dependency-diagram.html#gradle_generate

IntelliJ IDEA – adaugare dependinte in pom.xml (Proiect cu sistem build tip Maven)

<https://www.jetbrains.com/help/idea/work-with-maven-dependencies.html>

Configurare proiect cu JUnit 4 sau JUnit 5

<https://www.digitalocean.com/community/tutorials/junit-setup-maven>

Anotarile oferite de JUnit 5

<https://junit.org/junit5/docs/current/user-guide/#writing-tests-annotations>

Asserturile oferite de JUnit 5

<https://junit.org/junit5/docs/current/user-guide/#writing-tests-assertions>

IntelliJ IDEA – configurare repository GIT

<https://www.jetbrains.com/help/idea/set-up-a-git-repository.html>

Laborator 6

Hints.

6.8.1, 6.8.2:

În clasele `NewIntCalculatorTest`, `DoubleCalculatorTest` realizați testele unitare următoare (nu uitați să adăugați anotările aferente):

```
public void testAddPositive() {...}
public void testAddNegatives() {...}
public void testSubtractPositives() {...}
public void testSubtractNegatives() {...}
public void testMultiplyPositives() {...}
public void testMultiplyNegatives() {...}
public void testMultiplyBy0() {...}
public void testDividePositives() {...}
public void testDivideNegatives() {...}
public void testDivideBy0() {...}
```

6.8.3

```
class AplicatieCuBursaTest {
    AplicatieCuBursa appCuBursa = new AplicatieCuBursa();

    @Test
    void sortTest1() {
        //arrange - creati o lista de StudentBursieri - o puteti lua pe cea de mai sus

        //act - apelati appCuBursa.sort(lista)

        //assert - parcurgeti noua lista, si verificati 2 cate 2 studenti daca satisfac
        toate conditiile de comparare
    }
}
```